

1. What Spark Is

Spark is a distributed computation engine. Your code describes transformations. The driver builds and optimizes a plan. Executors run tasks over partitions. Spark is useful when parallelism, throughput, recovery, or intermediate scale justify coordination overhead.

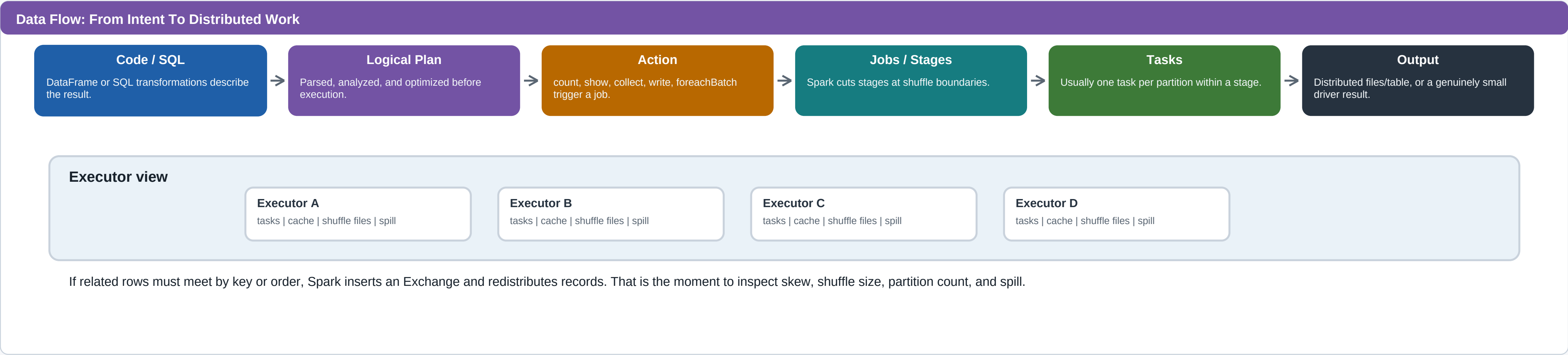
- Use it for repeatable bulk transformations and large analytical pipelines.
- Do not use it just because a cluster exists or the data feels big.
- The key question: what constraint requires distributed execution?

2. Runtime Actors

- Driver: creates the SparkSession, plans work, schedules tasks, tracks lineage, and receives small action results.
- Executors: run tasks, cache partitions, write shuffle files, spill to disk, and report progress.
- Cluster manager: grants resources. It does not replace the Spark driver.
- Partitions become the practical unit of parallel work inside a stage.

3. Cost Switches

- Lazy transformations are cheap descriptions until an action triggers execution.
- Narrow work can pipeline within a partition: select, filter, map-like projections.
- Wide work creates a shuffle: join, groupBy, distinct, global sort, many windows.
- Shuffles add serialization, network I/O, disk spill, coordination, and failure risk.



Common Healthy Patterns

- Read with explicit schemas for production data.
- Filter and select early so less data reaches joins and shuffles.
- Join with intent: broadcast the small side, repartition when key distribution matters, validate row counts.
- Persist only reused expensive intermediates, then materialize once.
- Write distributed outputs with sane file sizes, then compact intentionally when needed.
- Use EXPLAIN FORMATTED and Spark UI evidence before tuning.
- Keep large results distributed; bring only aggregate summaries or bounded samples to the driver.
- Reason from partition distribution, not only total row count or total data size.
- Tune the query shape first, then memory and executor sizing after evidence.
- Prefer simple, checkpointed pipelines when lineage becomes hard to inspect or retry.

Common Anti-Patterns

- collect(), toPandas(), or show() on data that is not proven small.
- coalesce(1) in production pipelines to force a single output file.
- Caching everything without reuse, materialization, or eviction discipline.
- Assuming more executors fix skew, bad joins, tiny files, or low task count.
- Ignoring Exchange, Sort, BroadcastExchange, and AdaptiveSparkPlan in EXPLAIN.
- Treating temp views as durable tables or SQL syntax as proof of cheap execution.
- Dropping repartition or cache calls into code without measuring the stage they affect.
- Joining before pruning columns or filtering rows that are irrelevant to the result.
- Confusing executor memory with driver memory; each process fails independently.
- Optimizing for one output file instead of healthy parallel writes and downstream compaction.